

NEX'SIRH GUINÉE — SaaS

Étapes de réalisation de A à Z

Guide d'exécution opérationnel — 26 étapes, du premier compte au lancement en production

Next.js · Vercel · Supabase · Stripe — vibe coding avec Antigravity & Claude Code

Version 1.0 — Juillet 2026

Compléments : Plan de réalisation (7 phases) · Maquettes app cliente · Maquettes console admin

Table des matières

Table des matières	2
Mode d'emploi de ce guide	3
Les 26 étapes de réalisation	4
Étape A — Prérequis et comptes.....	4
Étape B — Initialisation du dépôt et du projet Next.js.....	4
Étape C — Fichier de règles agents (CLAUDE.md) et intégration des maquettes.....	4
Étape D — Design system et composants de base.....	5
Étape E — Schéma fondation : entreprises, profils, rôles	5
Étape F — Authentification et écrans d'accès.....	5
Étape G — Row Level Security et tests d'isolation	6
Étape H — Layout applicatif (shell)	6
Étape I — Module Paramétrage (référentiels d'entreprise)	6
Étape J — Barèmes légaux versionnés (RTS, cotisations)	7
Étape K — Module Employés (le cœur RH)	7
Étape L — Import Excel/CSV des salariés	7
Étape M — Moteur de paie en TDD (tests d'abord).....	8
Étape N — Cycle de paie mensuel (brouillon → validé → clôturé)	8
Étape O — Bulletin PDF et journal de paie.....	8
Étape P — Documents administratifs (déclarations légales)	9
Étape Q — Module Congés et absences	9
Étape R — Temps, présence et heures supplémentaires.....	9
Étape S — Portail employé	10
Étape T — Documents RH et éditions	10
Étape U — Rapports et dashboard client.....	10
Étape V — Abonnements et facturation Stripe.....	10
Étape W — Console admin de la plateforme	11
Étape X — Emails, notifications et tâches planifiées.....	11
Étape Y — Durcissement : tests E2E, sécurité, performance, conformité	11
Étape Z — Migration des clients, documentation et lancement.....	12
Tableau de suivi A → Z	13
Jalons de démonstration	13

Mode d'emploi de ce guide

Ce document découpe la réalisation complète du SaaS en 26 étapes séquentielles, nommées A à Z. Chaque étape indique : son objectif, les actions concrètes, un prompt type à donner à l'agent IA quand c'est utile, le critère objectif de fin (« Terminé quand »), et le point de sauvegarde Git. Les étapes reprennent exactement le périmètre validé : le plan en 7 phases, les maquettes de l'application cliente et celles de la console administrateur, qui servent de spécification visuelle.

Les 5 règles de travail (à relire avant chaque session)

- **1. Contexte d'abord** : chaque étape référence les règles métier du fichier CLAUDE.md (étape C). Un agent sans contexte improvise ; un agent avec CLAUDE.md exécute.
- **2. Petites itérations** : une étape = une branche Git = une ou plusieurs sessions d'agent. Jamais « fais tout le module paie » en un prompt.
- **3. Tests avant code pour l'agent** : pour la paie (étape M), les tests des 8 bulletins GARAYA s'écrivent AVANT le code, et deviennent un garde-fou permanent en CI.
- **4. Revue humaine ciblée** : migrations SQL, politiques RLS, webhooks Stripe, calculs de paie = relecture humaine obligatoire. UI et formulaires = validation visuelle contre les maquettes suffit.
- **5. Le critère de fin est roi** : ne passez jamais à l'étape suivante si le critère « Terminé quand » n'est pas atteint. C'est lui qui empêche la dette de s'accumuler.

Vue d'ensemble : correspondance étapes ↔ phases ↔ durées

Étapes	Phase du plan	Contenu	Durée
A → D	Phase 0 — Mise en place	Comptes, projet, règles agents, design system	2,5 j
E → H	Phase 1 — Socle	Schéma, auth, RLS + isolation, layout	5 j
I → J	Phase 2 — Paramétrage	Référentiels, barèmes versionnés	3 j
K → L	Phase 3 — Personnel	Employés, mouvements, import Excel	4 j
M → P	Phase 4 — Paie 🎯	Moteur TDD, cycle, bulletins PDF, docs administratifs	7 j
Q → S	Phase 5 — Congés & portail	Congés, temps/HS, portail employé	5,5 j
T → X	Phase 6 — Produit complet	Documents, rapports, Stripe, console admin, automatisations	9 j
Y → Z	Phase 7 — Lancement	Durcissement, migration pilote, production	5 j

Total : 41 jours effectifs (fourchette du plan : 35–45 j), soit 8 à 10 semaines à raison de 2-3 h/jour.

Les 26 étapes de réalisation

Étape A — Prérequis et comptes

Phase 0 • Durée : 0,5 j

Objectif : Réunir tous les comptes et accès avant d'écrire la moindre ligne de code.

- Créer les comptes : GitHub, Vercel, Supabase (2 projets : nexsirh-dev et nexsirh-prod), Stripe (mode test), Resend, et réserver le domaine (ex. nexsirh.gn ou nexsirh.app).
- Installer localement : Node.js LTS, Git, Supabase CLI, Vercel CLI, Antigravity, Claude Code.
- Créer un coffre de secrets (gestionnaire de mots de passe) : aucune clé ne sera jamais committée.

✓ **Terminé quand :** tous les comptes sont accessibles et les CLI répondent (node -v, supabase -v, vercel -v, claude --version).

🔧 **Git :** —

Étape B — Initialisation du dépôt et du projet Next.js

Phase 0 • Durée : 0,5 j

Objectif : Créer le squelette du projet et la chaîne de déploiement continue.

- Créer le dépôt GitHub privé nexsirh, avec branche main protégée.
- npx create-next-app@latest (TypeScript, App Router, Tailwind, ESLint) ; ajouter shadcn/ui, Prettier, husky (lint au commit).
- Lier le dépôt à Vercel : chaque push sur main déploie en production, chaque branche crée une preview.
- Page d'accueil provisoire « Nex'SIRH — bientôt » déployée.

Prompt agent : Initialise un projet Next.js 15 App Router TypeScript avec Tailwind et shadcn/ui. Ajoute Prettier + ESLint strict + husky pre-commit. Structure de dossiers : app/(auth), app/(app), app/(portail), app/(admin), lib/, components/, supabase/migrations/.

✓ **Terminé quand :** l'URL Vercel affiche la page provisoire ; un push déclenche un déploiement automatique.

🔧 **Git :** tag v0.1-socle

Étape C — Fichier de règles agents (CLAUDE.md) et intégration des maquettes

Phase 0 • Durée : 0,5 j

Objectif : Donner aux agents IA (Antigravity, Claude Code) un contexte permanent : c'est LE fichier qui évite la dérive du vibe coding.

- Rédiger CLAUDE.md à la racine : stack, conventions de nommage, structure des dossiers, règles de paie complètes (chaîne de calcul, barème RTS 4 tranches, CNSS/VF/CFPA), interdits (jamais de barème en dur, jamais de service_role côté client, jamais modifier une migration appliquée).
- Copier les 2 fichiers de maquettes (app cliente + console admin) dans /design/ du dépôt : ce sont les références visuelles que les agents doivent reproduire.
- Extraire les tokens de design dans tailwind.config : vert #0F5C49, or #D9A441, encre #1C2422, polices Bricolage Grotesque / Public Sans / IBM Plex Mono.

✓ **Terminé quand** : CLAUDE.md relu et complet ; un agent interrogé sur « quel est le barème RTS ? » répond correctement sans chercher.

🔗 **Git** : commit « règles agents + design de référence »

Étape D — Design system et composants de base

Phase 0 • Durée : 1 j

Objectif : Construire une fois les briques UI réutilisées partout, à l'identique des maquettes.

- Composants : Badge de statut, Montant GNF (Plex Mono, séparateurs d'espace fin, entiers), Panel, KPI Card, Table triable, Modale, Toast, Rail latéral, Topbar.
- Page /design-system de démonstration (storybook maison) pour valider visuellement contre les maquettes HTML.
- Fonction utilitaire formatGNF(n) testée (3 745 000, jamais de décimales).

Prompt agent : Reproduis les composants du fichier /design/Maquettes_NexSIRH_SaaS.html en composants React + Tailwind + shadcn/ui : Badge, MontantGNF, Panel, KpiCard, DataTable, Modal, Toast. Fidélité visuelle stricte aux maquettes (couleurs, polices, rayons).

✓ **Terminé quand** : /design-system est visuellement conforme aux maquettes côte à côte.

🔗 **Git** : tag v0.2-design

Étape E — Schéma fondation : entreprises, profils, rôles

Phase 1 • Durée : 1 j

Objectif : Poser les tables socles du multi-tenant.

- Migration 001 : companies (raison sociale, NIF, n° CNSS, adresse, convention, statut, période d'essai), profiles (lié à auth.users : company_id, rôle, matricule lié), enum role (admin, rh, dg, manager, comptable, employe).
- Trigger : à l'inscription, création automatique du profil admin lié à l'entreprise.
- Table audit_log générique (acteur, action, cible, avant/après, IP) + triggers sur companies et profiles.

✓ **Terminé quand** : migration appliquée en dev via supabase db push ; insertion d'une entreprise de test réussie.

🔗 **Git** : migration 001 committée

Étape F — Authentification et écrans d'accès

Phase 1 • Durée : 1,5 j

Objectif : Reproduire les 3 écrans d'auth des maquettes avec Supabase Auth.

- Connexion, Inscription en 3 étapes (email → entreprise → compte admin), Mot de passe oublié — mêmes textes et mise en page que les maquettes.
- Middleware Next.js : routes /(app) inaccessibles sans session ; redirection vers /connexion.
- Emails transactionnels (confirmation, reset) via Resend, gabarits aux couleurs Nex'SIRH.

Prompt agent : Implémente l'authentification Supabase (email/mot de passe) avec les écrans de /design/Maquettes_NexSIRH_SaaS.html : connexion, inscription 3 étapes qui crée company + profile admin (transaction), mot de passe oublié. Middleware de protection des routes.

✓ **Terminé quand** : je peux créer une entreprise de bout en bout, me déconnecter, me reconnecter, réinitialiser mon mot de passe.

🔗 **Git** : tag v0.3-auth

Étape G — Row Level Security et tests d'isolation

Phase 1 · Durée : 1,5 j

Objectif : Le verrou du multi-tenant : aucune donnée ne doit fuir entre entreprises. Étape NON négociable avant de continuer.

- Politiques RLS sur toutes les tables : company_id = (profil connecté).company_id, affinées par rôle (le manager ne lit que son équipe, jamais les salaires ; l'employé ne lit que ses lignes).
- Fonction SQL helper current_company_id() + tests.
- Suite de tests d'isolation automatisée (Vitest + client Supabase) : entreprise A ne voit jamais B, en lecture ET en écriture, pour chaque rôle.
- REVUE HUMAINE ligne par ligne de chaque policy (règle du plan : jamais délégué à 100 % à l'agent).

✓ **Terminé quand** : la suite d'isolation passe (verte) et est branchée en CI GitHub Actions : elle bloquera tout merge futur.

🔗 **Git** : tag v0.4-rls – CI active

Étape H — Layout applicatif (shell)

Phase 1 · Durée : 1 j

Objectif : Le squelette visuel dans lequel tous les modules vont s'emboîter.

- Rail latéral vert (sections Gestion RH / Paie & finances / Administration), topbar avec recherche et cloche, zone de contenu — copie des maquettes.
- Affichage conditionnel des entrées de menu selon le rôle (matrice des droits).
- Layout distinct pour /(portail) (bandeau vert, 3 onglets) préparé pour l'étape S.

✓ **Terminé quand** : navigation fluide entre pages vides, menu correct pour chaque rôle testé (6 comptes de test).

🔗 **Git** : tag v0.5-shell – fin Phase 1

Étape I — Module Paramétrage (référentiels d'entreprise)

Phase 2 · Durée : 2 j

Objectif : Les tables de référence que tous les modules consomment.

- Migrations + écrans : Entreprise (infos légales), Départements, Postes (catégories Employé/Cadre/Cadre Sup.), Types de contrats (CDD ≤ 24 mois, essai), Types d'absences (CA 2,5 j/mois, CMAT 98 j...), Types de primes (imposable RTS ? soumise CNSS ?), Jours fériés guinéens 2026.
- Onglets du Paramétrage identiques aux maquettes ; seed automatique des valeurs guinéennes par défaut à la création d'une entreprise.

✓ **Terminé quand** : une nouvelle entreprise arrive avec ses référentiels pré-remplis et modifiables.

🔗 **Git** : tag v0.6-parametrage

Étape J — Barèmes légaux versionnés (RTS, cotisations)

Phase 2 • Durée : 1 j

Objectif : Le cœur réglementaire : barèmes datés, jamais écrasés.

- Tables globales (hors tenant) `tax_brackets` et `contribution_rates` avec date d'effet et référence de version (v2026.1).
- Seed : RTS 4 tranches (0–2 M : 5 % ; 2–5 M : 8 % ; 5–10 M : 10 % ; > 10 M : 15 %), CNSS 5/18 % plafond 2 500 000, VF 6 %, CFPA 1,5 %.
- Fonction `getBaremeAt(date)` qui retourne la version applicable à une date donnée.

✓ **Terminé quand :** tests unitaires : `getBaremeAt` retourne la bonne version pour des dates avant/après un changement.

🔗 **Git :** tag `v0.7-baremes`

Étape K — Module Employés (le cœur RH)

Phase 3 • Durée : 3 j

Objectif : CRUD complet des salariés, conforme aux maquettes (liste, fiche 6 onglets, assistant de création, mouvements).

- Migration employees (tous les champs de la fiche : identité, coordonnées, pièce, professionnel, contrat, salaire + primes fixes, banque, statut) + `employee_movements`.
- Écrans : liste filtrable (Tous/Actifs/CDD/Essai/Sortis), fiche 6 onglets, assistant « Nouvel employé » en 6 étapes, modale Modifier (hors salaire), modale Mouvement (augmentation, promotion, départ — avec ancienne/nouvelle valeur tracée).
- Validations métier : âge ≥ 16 ans, matricule EMP-XXX auto, CDD ≤ 24 mois, date de fin obligatoire si CDD.
- Alertes calculées : CDD à 30 j, pièce expirant, fin d'essai (affichées sur le dashboard).

Prompt agent : Implémente le module Employés conformément aux maquettes (vues `v-employees`, `v-fiche`, modales `mNouvelEmploye`, `mModifEmploye`, `mMouvement`). Toute modification de salaire passe par un mouvement journalisé — jamais d'édition directe.

✓ **Terminé quand :** cycle complet : créer → modifier → augmenter (mouvement tracé) → faire sortir un employé ; alertes visibles.

🔗 **Git :** tag `v0.8-employees`

Étape L — Import Excel/CSV des salariés

Phase 3 • Durée : 1 j

Objectif : Reprendre en 5 minutes les données des clients venant du fichier `.xlsm`.

- Assistant d'import : dépôt du fichier, mapping des colonnes détecté automatiquement, prévisualisation, rapport d'erreurs ligne par ligne (âge, doublons matricule, montants non entiers).
- Gabarit Excel téléchargeable pré-formaté (mêmes colonnes que `DATA_Employes` de l'ancien SIRH).

✓ **Terminé quand :** import réussi d'un fichier de 24 salariés type GARAYA avec 2 erreurs volontaires correctement signalées.

🔗 **Git :** tag `v0.9-import`

Étape M — Moteur de paie en TDD (tests d'abord)

Phase 4 • Durée : 2 j

Objectif : L'étape la plus critique du projet : le calcul, écrit à partir des tests, jamais l'inverse.

- ÉCRIRE D'ABORD les tests : les 8 bulletins GARAYA HOLDING saisis en fixtures (entrées : salaire + primes ; sorties attendues : brut, base CNSS, CNSS 5 %/18 %, net imposable, RTS, VF, CFPA, net) — conformité au franc près exigée.
- Puis implémenter lib/paie/ : fonctions pures calculBrut, baseCNSS, cotisationsCNSS, netImposable (déduction des indemnités exonérées), calculRTS (tranche par tranche via getBaremeAt), calculVF, calculCFPA, netAPayer.
- Cas limites testés : brut sous le plafond CNSS (SYLLA : base = brut 2 215 000), franchissement de tranche RTS (PLEGNEMOU : 5 % + 8 %), prêt personnel (FAYE : net 1 503 268).

Prompt agent : Voici 8 fixtures de bulletins réels (fichier tests/paie/garaya.fixtures.ts). Écris les tests Vitest qui vérifient chaque montant, PUIS implémente lib/paie jusqu'à ce que tout passe. Interdiction de modifier les fixtures. Barèmes lus depuis la base, jamais codés en dur.

✓ **Terminé quand** : npm test : 8/8 bulletins conformes au franc près. Ce jeu de tests devient le garde-fou permanent de la CI.

🔗 **Git** : tag v1.0-moteur-paie 🔄

Étape N — Cycle de paie mensuel (brouillon → validé → clôturé)

Phase 4 • Durée : 2 j

Objectif : L'orchestration qui corrige le défaut historique de la version Excel (recalcul/suppression impossibles).

- Migrations payroll_runs + payslips (montants figés à la clôture) ; états : brouillon (recalcul/suppression OK) → validé → clôturé (verrouillage définitif, trigger SQL qui refuse tout UPDATE).
- Écran Paie des maquettes : sélecteur de période, 4 KPIs, table des bulletins, boutons Recalculer (modale), Clôturer (modale avec saisie « CLOTURER »).
- Éléments variables : avances, prêts avec échéancier, autres primes/retenues ; rappels/reprises pour régulariser un mois clôturé.
- Saisie des retenues conservée lors d'un recalcul (comme promis dans la modale des maquettes).

✓ **Terminé quand** : générer → modifier un salaire → recalculer → clôturer → vérifier qu'un UPDATE direct en SQL est rejeté.

🔗 **Git** : tag v1.1-cycle-paie

Étape O — Bulletin PDF et journal de paie

Phase 4 • Durée : 1,5 j

Objectif : Les documents que le client imprime chaque mois.

- Génération PDF serveur du bulletin au format GARAYA (en-tête entreprise, gains, cotisations avec parts salariale/patronale, cumuls annuels, net encadré) — stocké dans Supabase Storage, listé dans Documents.
- Journal de paie mensuel (une ligne par salarié + totaux) en PDF et export Excel.

- Génération en lot (ZIP des 24 bulletins) et envoi par email au salarié.

✓ **Terminé quand** : le PDF de FAYE juin 2026 est visuellement comparable au bulletin Sage d'origine ; totaux du journal = somme des bulletins.

🔗 **Git** : tag v1.2-bulletins

Étape P — Documents administratifs (déclarations légales)

Phase 4 • Durée : 1,5 j

Objectif : Le module validé sur les maquettes : les documents exigibles par l'État guinéen.

- Générateur avec sélection période + cases à cocher, comme la modale mDocsAdmin : Registre du personnel (Inspection du Travail/CNSS), Déclaration CNSS mensuelle, État RTS mensuel au format eTax/DNI (sur le modèle de la déclaration ENGUITRACI), État des salaires, Suivi des congés, Fiche individuelle (avec sélecteur d'employé).
- Tous archivés automatiquement dans Documents avec horodatage et auteur.

✓ **Terminé quand** : les 6 documents se génèrent pour Mai 2026 et l'état RTS totalise exactement la colonne RTS du journal de paie.

🔗 **Git** : tag v1.3-docs-admin – fin Phase 4

Étape Q — Module Congés et absences

Phase 5 • Durée : 2,5 j

Objectif : Workflow complet à deux niveaux, planning, soldes conformes au Code du travail.

- Migrations leave_requests + leave_balances ; calcul des jours ouvrables (week-ends + fériés guinéens exclus) ; droits 2,5 j/mois + 1 j par tranche de 5 ans d'ancienneté ; report N-1.
- Écrans des maquettes : onglets À traiter / Approuvées / Refusées / Planning coloré ; modales demande (solde simulé avant envoi) et validation (Manager puis RH, motif obligatoire au refus, alerte de chevauchement d'équipe).
- Emails de notification à chaque transition ; absences non rémunérées transmises à la paie.

✓ **Terminé quand** : cycle complet demande → validation manager → validation RH → solde décrétement → certificat de congé générable.

🔗 **Git** : tag v1.4-conges

Étape R — Temps, présence et heures supplémentaires

Phase 5 • Durée : 1,5 j

Objectif : Feuilles de temps alimentant automatiquement la paie.

- Migration timesheets/entries ; saisie journalière (arrivée/départ), base 173,33 h/mois, 40 h/semaine.
- Calcul automatique des majorations : +25 % (8 premières HS), +50 % au-delà, +100 % dimanches et fériés — montants injectés dans le bulletin du mois.
- Validation par le manager avant transmission à la paie (bouton des maquettes).

✓ **Terminé quand** : le cas CONTE (8 h à 25 %, 4 h à 50 %, 3 h à 100 % = 254 810 GNF) est reproduit exactement dans son bulletin.

🔗 **Git** : tag v1.5-temps

Étape S — Portail employé

Phase 5 • Durée : 1,5 j

Objectif : La grande nouveauté vs Excel : le libre-service salarié.

- Layout portail des maquettes (bandeau vert, Accueil / Mes bulletins / Mes congés) ; rôle employé strictement limité à ses propres données (policies RLS déjà prêtes, à tester).
- Invitation des salariés par email depuis leur fiche ; téléchargement des bulletins PDF ; demande de congé en ligne entrant dans le workflow.

✓ **Terminé quand :** Bountouraby (compte employé) voit ses 3 bulletins, son solde 12,5 j, dépose une demande — et ne peut accéder à rien d'autre (test d'isolation).

🔗 **Git :** tag v1.6-portail — fin Phase 5

Étape T — Documents RH et éditions

Phase 6 • Durée : 1,5 j

Objectif : Compléter la panoplie documentaire.

- Attestation de travail, certificat de travail, certificat de congé, contrat de travail pré-rempli (CDI/CDD), lettres types.
- Solde de tout compte automatique : congés non pris payés, indemnité de licenciement selon ancienneté (Code du travail), préavis, paie au prorata — avec feuille de calcul détaillée jointe.
- Modale « Générer un document » des maquettes (type, salarié, signataire, mention).

✓ **Terminé quand :** un départ de salarié produit un solde de tout compte cohérent, vérifié à la main sur un cas.

🔗 **Git :** tag v1.7-documents

Étape U — Rapports et dashboard client

Phase 6 • Durée : 1,5 j

Objectif : Le pilotage pour le DG et le RH.

- Dashboard d'accueil des maquettes : 4 KPIs, masse salariale 6 mois (Recharts), alertes RH, congés à traiter, donut effectif.
- Page Rapports : coût employeur mensuel, indicateurs annuels (absentéisme, turnover, ancienneté, cumuls RTS/CNSS), 4 états imprimables, exports Excel/CSV, source de données Power BI (lien vers vues SQL en lecture).

✓ **Terminé quand :** les chiffres du dashboard = requêtes SQL vérifiées à la main sur les données de test.

🔗 **Git :** tag v1.8-rapports

Étape V — Abonnements et facturation Stripe

Phase 6 • Durée : 2 j

Objectif : Transformer le produit en business.

- Plans Starter/Business/Cabinet en base + Stripe ; essai 30 jours sans carte ; page Abonnement des maquettes (bannière essai, 3 cartes, modale de paiement).

- Webhooks Stripe → table subscriptions (trial, active, past_due, canceled) ; politique : lecture seule à j+15 d'impayé, jamais de suppression de données ; factures PDF.
- Limites par plan appliquées (15/50 salariés, nb d'utilisateurs) avec messages d'upgrade élégants.
- Virement manuel possible (activation par la console admin) pour le marché local.

✓ **Terminé quand** : parcours test complet : essai → carte test Stripe → facture → simulation d'échec → lecture seule → régularisation.

🔗 **Git** : tag v1.9-abonnements – fin Phase 6

Étape W — Console admin de la plateforme

Phase 6+ · Durée : 3 j

Objectif : Construire le back-office maquetté (thème anthracite/or) qui pilote tout.

- Rôle super-admin hors tenant + layout /(admin) ; connexion séparée avec 2FA.
- Vue d'ensemble (MRR, actifs, essais, churn, santé, à traiter) ; Entreprises + fiche (usage, notes, journal) ; Abonnements & impayés (relances) ; Plans & codes promo ; Barèmes légaux avec publication versionnée ET simulation obligatoire sur les 8 bulletins GARAYA avant publication ; Support (tickets) ; Annonces (bannière + email ciblé) ; Monitoring (crons, incidents) ; Audit append-only ; Équipe & API.
- Accès support (impersonation) : consentement client requis, 60 min max, lecture seule sur salaires, session tracée dans l'audit des deux côtés.
- Suspension d'entreprise : lecture seule, données intactes, réversible.

Prompt agent : Implémente la console admin conformément à /design/Console_Admin_NexSIRH.html. Règles de gouvernance impératives : publication de barème bloquée si la simulation sur les fixtures GARAYA échoue ; impersonation avec consentement + journalisation double ; seul le super-admin suspend.

✓ **Terminé quand** : toutes les vues et modales de la maquette fonctionnent sur données réelles ; l'impersonation apparaît dans le journal du client.

🔗 **Git** : tag v2.0-console

Étape X — Emails, notifications et tâches planifiées

Phase 6+ · Durée : 1 j

Objectif : L'automatisation qui fait vivre le produit sans intervention.

- Crons Vercel : alertes RH quotidiennes (06:00), rappels essais expirants (08:00), relances impayés (09:00), vérification sauvegardes (04:30).
- Gabarits email Resend : invitation, congé à valider/validé/refusé, bulletin disponible, essai J-5/J-1, facture, incident.
- Page de préférences de notifications par utilisateur.

✓ **Terminé quand** : chaque cron s'exécute en dev avec logs propres ; un essai fictif à J-5 reçoit son email.

🔗 **Git** : tag v2.1-automations

Étape Y — Durcissement : tests E2E, sécurité, performance, conformité

Phase 7 · Durée : 3 j

Objectif : Passer de « ça marche » à « c'est fiable ».

- Tests E2E Playwright des parcours critiques : inscription → import → paie → clôture → bulletin PDF ; congé bout en bout ; impayé → lecture seule.
- Cas limites du plan : embauche en cours de mois (prorata), congé à cheval sur deux mois, fin de CDD pendant un congé, mois de février, 50 salariés (charge).
- Revue de sécurité finale : relecture de TOUTES les politiques RLS, en-têtes HTTP, rate limiting auth, dépendances (npm audit), rotation des clés.
- Conformité données : export RGPD par entreprise, mentions légales, CGU/CGV, politique de confidentialité, page de statut publique.
- Sauvegardes : test de restauration RÉEL sur le projet dev (une sauvegarde non testée n'existe pas).

✓ **Terminé quand** : suite E2E verte en CI ; restauration réussie chronométrée ; checklist sécurité signée.

🔗 **Git** : tag v2.2-rc1 (release candidate)

Étape Z — Migration des clients, documentation et lancement

Phase 7 • Durée : 2 j

Objectif : Mettre le produit entre les mains des clients.

- Migrer le premier client pilote (données GARAYA-like) : import employés + soldes de congés + cumuls de paie de l'année ; première paie réelle en accompagnement, comparée à l'ancien fichier Excel.
- Documentation : guide RH illustré, guide manager, guide employé (portail), base de connaissances (3 articles prioritaires : import, comprendre la RTS, clôturer/régulariser).
- Page d'accueil marketing (offres, essai gratuit) ; bascule DNS du domaine ; Stripe en mode live ; monitoring d'alerte (email si erreur critique).
- Formation du client pilote (2 h) puis ouverture des inscriptions publiques. 🚀

✓ **Terminé quand** : le client pilote a clôturé sa première paie réelle sur Nex'SIRH, identique au franc près à son ancien fichier.

🔗 **Git** : tag v1.0.0 – PRODUCTION 🎉

Tableau de suivi A → Z

À imprimer ou copier dans un outil de suivi. Ne cochez une étape que si son critère « Terminé quand » est rempli.

Ét.	Intitulé	Durée	Statut	Début	Fin
A	Prérequis et comptes	0,5 j	☐		
B	Initialisation du dépôt et du projet Next.js	0,5 j	☐		
C	Fichier de règles agents (CLAUDE.md) et intégration des maquettes	0,5 j	☐		
D	Design system et composants de base	1 j	☐		
E	Schéma fondation : entreprises, profils, rôles	1 j	☐		
F	Authentification et écrans d'accès	1,5 j	☐		
G	Row Level Security et tests d'isolation	1,5 j	☐		
H	Layout applicatif (shell)	1 j	☐		
I	Module Paramétrage (référentiels d'entreprise)	2 j	☐		
J	Barèmes légaux versionnés (RTS, cotisations)	1 j	☐		
K	Module Employés (le cœur RH)	3 j	☐		
L	Import Excel/CSV des salariés	1 j	☐		
M	Moteur de paie en TDD (tests d'abord)	2 j	☐		
N	Cycle de paie mensuel (brouillon → validé → clôturé)	2 j	☐		
O	Bulletin PDF et journal de paie	1,5 j	☐		
P	Documents administratifs (déclarations légales)	1,5 j	☐		
Q	Module Congés et absences	2,5 j	☐		
R	Temps, présence et heures supplémentaires	1,5 j	☐		
S	Portail employé	1,5 j	☐		
T	Documents RH et éditions	1,5 j	☐		
U	Rapports et dashboard client	1,5 j	☐		
V	Abonnements et facturation Stripe	2 j	☐		
W	Console admin de la plateforme	3 j	☐		
X	Emails, notifications et tâches planifiées	1 j	☐		
Y	Durcissement : tests E2E, sécurité, performance, conformité	3 j	☐		
Z	Migration des clients, documentation et lancement	2 j	☐		

Légende : ☐ À faire — ● En cours — ☒ Terminé.

Jalons de démonstration

Quatre moments où montrer le produit (au client pilote, à un associé, ou pour se motiver) :

- **Fin d'étape H (≈ semaine 2)** : le fichier s'ouvre sur une vraie page de connexion, deux entreprises étanches. Le socle est solide.
- **Fin d'étape O (≈ semaine 5)** : LA démo qui compte : une paie GARAYA calculée au franc près, bulletin PDF à l'appui.
- **Fin d'étape V (≈ semaine 7-8)** : produit vendable : essai, paiement, portail employé, documents légaux.
- **Étape Z (≈ semaine 9-10)** : premier client en production réelle. 🚀

Document établi le 12 juillet 2026. Prochain geste concret : ouvrir l'étape A et créer les comptes — 30 minutes suffisent pour démarrer.